

Product Requirements Document (PRD)

Product: Personal Branding & Publishing Portal

Architecture: Next.js (Frontend), Supabase (Per-Client Database), Clerk (Authentication), Tailwind CSS (Styling)

1. Product Overview

A highly customizable, individually hosted personal branding platform designed to serve as both a professional portfolio and an active lead-generation engine. Each client receives a dedicated instance with an isolated Supabase database. The platform allows professionals to showcase dual-track expertise, highlight case studies, embed media, and operate a hybrid newsletter/blog publishing system to capture and engage an audience.

2. User Roles

- **Site Owner (Admin):** Authenticated via Clerk. Has full access to a gated dashboard to write newsletters, publish content, manage portfolio items, and view subscriber metrics.
-
- **Visitor (Public):** Unauthenticated users who can view the portfolio, read blog posts, subscribe to the newsletter, and click outbound links (courses, SaaS platforms, social media).
-

3. Core Features & Functionalities

3.1 Newsletter & Publishing Engine

The site functions as an active growth engine rather than a static digital business card.

- **Unified Publishing:** When the Site Owner publishes a newsletter, the system simultaneously generates a public-facing blog post on the site (for SEO) and triggers an email dispatch to subscribers.
-
- **Rich Text Editor:** Admin dashboard requires a Markdown or WYSIWYG editor to draft architecture teardowns, "build in public" updates, and micro-lessons.
-
- **Subscriber Capture:** Prominent opt-in forms offering a lead magnet (e.g., cheat sheets or checklists) to capture visitor emails.
-
- **Email Dispatch Integration:** An integration with an Email Service Provider (ESP) like Resend or SendGrid is required to handle the actual bulk email delivery, triggered by a Next.js API route.
-

3.2 Dynamic Front-End Architecture

The public-facing site must structurally support the client's authority and conversion goals.

Component	Purpose	Example Content Mapping
Hero Section	Above-the-fold value proposition	Dual-strength positioning (e.g., Data Pipelines & Scalable Web Platforms). Clear primary CTA.
Stack Grid	Visual separation of capabilities	Group 1: Data Engineering (Databricks, PySpark). Group 2: Web (Next.js, Supabase).
Portfolio Showcase	Differentiate owned vs. client work	Split view: SaaS Products (Datavvy, Krosskut.io) vs. Bespoke Builds (Factory Digitization).
Education Hub	Monetization and authority	Prominent links to instructional content (AI orchestration, zero-code MVP courses).
Media Gallery	Trust building and communication	Embedded high-quality video content, technical walkthroughs, and social media reels.
Social & Links	Omnichannel connectivity	Footer and header routing to LinkedIn, YouTube, X, GitHub, and direct contact forms.

3.3 Admin Dashboard Capabilities

- **Content Management:** CRUD operations for Portfolio items, Case Studies, and Educational links.
-
- **Subscriber Management:** View active subscriber counts, export email lists, and manage unsubscribes.
-

- **Analytics Overview:** Basic tracking of newsletter open rates and top-performing blog posts.
-

4. Database Architecture (Supabase)

To support the independent client hosting model, each Supabase instance requires the following core tables:

Table Name	Description	Key Fields
<code>subscribers</code>	Users who opted into the newsletter	<code>id</code> , <code>email</code> , <code>status</code> (active/unsubscribed), <code>created_at</code>
<code>posts</code>	Newsletter issues doubling as blog posts	<code>id</code> , <code>title</code> , <code>slug</code> , <code>html_content</code> , <code>is_published</code> , <code>published_at</code>
<code>portfolio_items</code>	Projects, case studies, and SaaS products	<code>id</code> , <code>type</code> (owned vs client), <code>title</code> , <code>description</code> , <code>tech_stack</code> , <code>url</code>
<code>site_settings</code>	Global configurations for the client	<code>id</code> , <code>hero_headline</code> , <code>social_links</code> , <code>theme_colors</code>

5. Security & Performance

- **Authentication:** Clerk handles all identity management, protecting the `/admin` routes via Next.js middleware.
-
- **Row Level Security (RLS):** Supabase RLS policies must enforce that public users can only `SELECT` published posts and portfolio items, while only the authenticated admin can `INSERT`, `UPDATE`, or `DELETE`.
-
- **SEO Optimization:** Blog posts must utilize Next.js dynamic routing (`/blog/[slug]`) with server-side generated metadata tags for optimal search engine indexing.